

Dependency Parsing

Chapters 19.2 & 19.3 & 19.4

Nhóm 3

Phạm Ngọc Phú Thịnh Phạm Nguyễn Duy Anh
Nguyễn Trường Giang Hà Bùi Trọng Nghĩa

Khoa Khoa học Máy tính
Trường Đại học Công nghệ Thông tin - UIT **Xử lý ngôn ngữ tự nhiên**

Ngày 18 tháng 5 năm 2026

Nội dung bài thuyết trình

- 1 19.2.2 Feature-Based Classifier
- 2 19.2.3 Neural Classifier
- 3 19.2.4 Advanced Methods in Transition-Based Parsing
- 4 19.3 Graph-Based Dependency Parsing
 - Parsing via finding the maximum spanning tree
 - A feature-based algorithm for assigning scores
 - Neural Algorithm for Assigning Scores
- 5 19.4 Evaluation
- 6 19.5 Summary

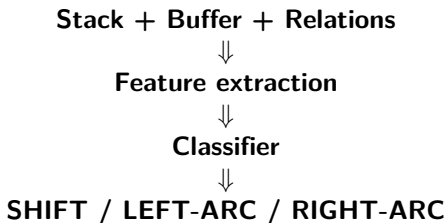
Table of Contents

- 1 19.2.2 Feature-Based Classifier
- 2 19.2.3 Neural Classifier
- 3 19.2.4 Advanced Methods in Transition-Based Parsing
- 4 19.3 Graph-Based Dependency Parsing
 - Parsing via finding the maximum spanning tree
 - A feature-based algorithm for assigning scores
 - Neural Algorithm for Assigning Scores
- 5 19.4 Evaluation
- 6 19.5 Summary

Feature-Based Classifier là gì?

Ý tưởng chính: xem việc chọn transition như một bài toán phân loại.

- Input là **configuration** hiện tại.
- Output là transition tiếp theo.
- Classifier học từ các ví dụ trong tập train.



Các loại feature thường dùng

Feature được lấy từ **configuration** hiện tại của parser.

Feature	Ý nghĩa
Word form	Dạng từ thật trong câu
Lemma	Dạng gốc của từ
POS tag	Từ loại của từ
Head	Từ đứng đầu trong quan hệ dependency
Dependency relation	Nhãn quan hệ phụ thuộc
Morphological feature	Đặc điểm ngữ pháp của từ

Các feature hữu ích thường đến từ: đỉnh stack, đầu buffer, và các relation đã tạo.

Feature Template

Feature template là khuôn mẫu dùng để sinh feature cụ thể.

Ký hiệu	Ý nghĩa
---------	---------

s	stack
-----	-------

b	buffer
-----	--------

s_1	từ trên đỉnh stack
-------	--------------------

s_2	từ ngay dưới s_1
-------	--------------------

b_1	từ đầu buffer
-------	---------------

w	word form
-----	-----------

t	POS tag
-----	---------

wt	word + POS tag
------	----------------

Ví dụ:

$s_1.w$

nghĩa là:

word form của từ trên đỉnh stack

Nếu $s_1 = \textit{flights}$, thì:

$s_1.w = \textit{flights}$

Các template cơ bản

Các template cơ bản dựa trên một vị trí trong configuration:

Template	Thông tin được lấy
$s_1.w$	Word form của đỉnh stack
$s_2.w$	Word form của từ dưới đỉnh stack
$s_1.t$	POS tag của đỉnh stack
$s_2.t$	POS tag của từ dưới đỉnh stack
$b_1.w$	Word form của đầu buffer
$b_1.t$	POS tag của đầu buffer
$s_1.wt$	Word form + POS tag của s_1

Các template này giúp parser biến configuration thành input cho classifier.

Ví dụ configuration

Xét configuration trong câu:

United canceled the morning flights to Houston

Stack

[root, canceled, flights]

Buffer

[to, Houston]

Relations

canceled → United

flights → morning

flights → the

Từ configuration này:

$s_1 = \text{flights}, \quad s_2 = \text{canceled}, \quad b_1 = \text{to}$

Transition đúng là:

SHIFT

Áp dụng feature template

Từ configuration trước, ta sinh các feature cụ thể:

Template	Feature cụ thể	Ý nghĩa
$s_1.w$	flights	word của s_1
$s_2.w$	canceled	word của s_2
$s_1.t$	NNS	POS tag của flights
$s_2.t$	VBD	POS tag của canceled
$b_1.w$	to	word của b_1
$b_1.t$	TO	POS tag của to
$s_1.wt$	flightsNNS	word + POS tag của s_1

Trong tập train, các feature này đi kèm nhãn đúng:

$$op = SHIFT$$

Vì sao transition đúng là SHIFT?

Configuration hiện tại:

$$Stack = [root, canceled, flights]$$
$$Buffer = [to, Houston]$$

- Cụm *to Houston* vẫn chưa được xử lý.
- Từ *Houston* chưa nằm trên stack.
- Parser cần đưa *to* từ buffer lên stack.

Vì vậy, parser chọn:

SHIFT

Nếu tạo arc quá sớm, parser có thể mất cơ hội xử lý dependent phía sau của *flights*.

Feature kết hợp

Feature đơn lẻ đôi khi chưa đủ thông tin.

Ví dụ:

$$s_1.t = NNS$$

chỉ cho biết từ trên đỉnh stack là danh từ số nhiều.

Feature kết hợp nối nhiều thông tin lại:

$$s_1.t \circ s_2.t = NNSVBD$$

- $s_1.t$: POS tag của từ trên đỉnh stack.
- $s_2.t$: POS tag của từ dưới đỉnh stack.
- $\circ$: phép nối feature.

Vì sao feature kết hợp quan trọng?

Transition thường phụ thuộc vào quan hệ giữa hai từ.

Ví dụ:

canceled flights

- *canceled*: động từ quá khứ, POS tag là VBD.
- *flights*: danh từ số nhiều, POS tag là NNS.

Feature kết hợp:

$$s_2.t \circ s_1.t = VBDNNS$$

Feature này giúp classifier nhận ra pattern:

động từ quá khứ + danh từ số nhiều

Pattern này có thể gợi ý quan hệ động từ – tân ngữ.

Table of Contents

- 1 19.2.2 Feature-Based Classifier
- 2 19.2.3 Neural Classifier
- 3 19.2.4 Advanced Methods in Transition-Based Parsing
- 4 19.3 Graph-Based Dependency Parsing
 - Parsing via finding the maximum spanning tree
 - A feature-based algorithm for assigning scores
 - Neural Algorithm for Assigning Scores
- 5 19.4 Evaluation
- 6 19.5 Summary

Neural Classifier: Kiến trúc tổng quát

Trong phần này, **oracle** được cài đặt bằng một neural network.

Thay vì dùng feature thủ công như:

$$s_1.w, \quad s_2.t, \quad b_1.w$$

mô hình dùng **vector representation** của các từ.

Các vị trí quan trọng:

- s_1 : từ trên đỉnh stack.
- s_2 : từ dưới s_1 .
- b_1 : từ đầu buffer.

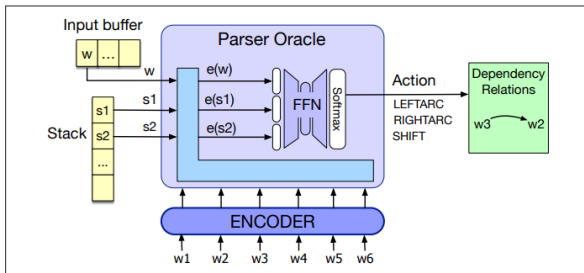


Figure 19.8 Neural classifier for the oracle for the transition-based parser. The parser takes the top 2 words on the stack and the first word of the buffer, represents them by their encodings (from running the whole sentence through the encoder), concatenates the embeddings and passes through a softmax to choose a parser action (transition).

Hình 19.8: Neural oracle trong transition-based parser

Neural Classifier: Cách chọn transition

Toàn bộ câu được đưa qua **encoder**:

$$w_1, \dots, w_n \rightarrow e(w_1), \dots, e(w_n)$$

Parser lấy ba vector chính:

$$e(s_1), \quad e(s_2), \quad e(b_1)$$

Sau đó nối chúng lại:

$$x = [e(b_1); e(s_1); e(s_2)]$$

Vector x được đưa vào:

$$FFN \rightarrow Softmax$$

Input sentence $\rightarrow$ **Encoder** $\rightarrow$ **FFN** $\rightarrow$ **Softmax** $\rightarrow$ **Transition**

Softmax tạo xác suất cho các transition:

Transition	Xác suất
SHIFT	0.70
LEFT-ARC	0.20
RIGHT-ARC	0.10

SHIFT

Chọn transition có xác suất cao nhất.

Table of Contents

- 1 19.2.2 Feature-Based Classifier
- 2 19.2.3 Neural Classifier
- 3 19.2.4 Advanced Methods in Transition-Based Parsing
- 4 19.3 Graph-Based Dependency Parsing
 - Parsing via finding the maximum spanning tree
 - A feature-based algorithm for assigning scores
 - Neural Algorithm for Assigning Scores
- 5 19.4 Evaluation
- 6 19.5 Summary

Vấn đề của greedy parsing

Greedy parsing

- Mỗi bước chỉ chọn một transition tốt nhất.
- Parser đi tiếp ngay sau khi chọn.
- Không xét các hướng parse khác.

Hạn chế

- Nếu chọn sai, parser khó sửa lại.
- Lỗi ở bước đầu có thể kéo theo lỗi sau.
- Cần phương pháp linh hoạt hơn.

Mục tiêu: cải thiện transition-based parsing bằng arc-eager và beam search.

Các transition trong Arc-eager

Transition	Ý nghĩa
SHIFT	Đưa từ đầu buffer lên stack.
LEFT-ARC	Tạo dependency: buffer front $\rightarrow$ stack top sau đó pop stack top.
RIGHT-ARC	Tạo dependency: stack top $\rightarrow$ buffer front sau đó đưa buffer front lên stack.
REDUCE	Pop từ trên đỉnh stack khi từ đó đã xử lý xong.

Arc-eager transition system

Arc-standard

- Tạo arc giữa hai phần tử trên cùng của stack.
- Khi một từ được gán head, nó bị pop khỏi stack.

Arc-eager

- Tạo arc giữa top stack và front buffer.
- Có thể tạo dependency sớm hơn.
- Thêm transition **REDUCE**.

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, the, flight, through, houston]	RIGHTARC	(root → book)
1	[root, book]	[the, flight, through, houston]	SHIFT	
2	[root, book, the]	[flight, through, houston]	LEFTARC	(the ← flight)
3	[root, book]	[flight, through, houston]	RIGHTARC	(book → flight)
4	[root, book, flight]	[through, houston]	SHIFT	
5	[root, book, flight, through]	[houston]	LEFTARC	(through ← houston)
6	[root, book, flight]	[houston]	RIGHTARC	(flight → houston)
7	[root, book, flight, houston]	[]	REDUCE	
8	[root, book, flight]	[]	REDUCE	
9	[root, book]	[]	REDUCE	
10	[root]	[]	Done	

Figure 19.9 A processing trace of *Book the flight through Houston* using the arc-eager transition operators.

Arc-standard vs Arc-eager

Ví dụ:

Book the flight through Houston

Dependency đúng:

book $\rightarrow$ flight

flight $\rightarrow$ Houston

Arc-standard

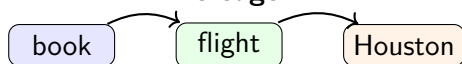


flight bị pop khỏi stack

không thể tạo

flight $\rightarrow$ Houston

Arc-eager



flight vẫn còn trên stack

nên tiếp tục tạo được

flight $\rightarrow$ Houston

Arc-eager giữ dependent trên stack lâu hơn, nên xử lý tốt hơn các trường hợp cần gắn thêm dependent phía sau.

Ý nghĩa của trace arc-eager

Book the flight through Houston

Các quan hệ quan trọng

root → *book*

book → *flight*

flight → *Houston*

flight → *the*

Houston → *through*

Điểm chính

- Arc-eager tạo *book* → *flight* sớm.
- *flight* vẫn được giữ trên stack.
- Sau đó *flight* vẫn có thể làm head của *Houston*.
- REDUCE dùng để bỏ từ khi đã xử lý xong.

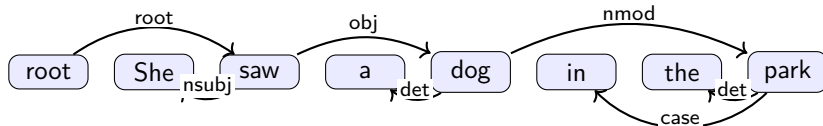
Arc-eager = tạo quan hệ sớm hơn nhưng vẫn giữ khả năng xử lý dependent sau.

Bài tập: Arc-eager

Câu ví dụ:

She saw a dog in the park

Cây dependency đúng:



Câu hỏi

Hãy tìm chuỗi transition theo **Arc-eager** để tạo đúng cây dependency trên.

Lời giải: Arc-eager

Bước	Stack	Buffer	Transition	Arc
0	[root]	[She, saw, a, dog, in, the, park]	SHIFT	–
1	[root, She]	[saw, a, dog, in, the, park]	LEFT-ARC	saw → She
2	[root]	[saw, a, dog, in, the, park]	RIGHT-ARC	root → saw
3	[root, saw]	[a, dog, in, the, park]	SHIFT	–
4	[root, saw, a]	[dog, in, the, park]	LEFT-ARC	dog → a
5	[root, saw]	[dog, in, the, park]	RIGHT-ARC	saw → dog
6	[root, saw, dog]	[in, the, park]	SHIFT	–
7	[root, saw, dog, in]	[the, park]	SHIFT	–
8	[root, saw, dog, in, the]	[park]	LEFT-ARC	park → the
9	[root, saw, dog, in]	[park]	LEFT-ARC	park → in
10	[root, saw, dog]	[park]	RIGHT-ARC	dog → park
11	[root, saw, dog, park]	[]	REDUCE	–
12	[root, saw, dog]	[]	REDUCE	–
13	[root, saw]	[]	REDUCE	–
14	[root]	[]	DONE	–

Beam search: Ý tưởng chính

Greedy

- Giữ 1 configuration tốt nhất.
- Nhanh.
- Dễ sai nếu chọn nhầm sớm.

$$\hat{T}(c) = \arg \max_t \text{Score}(t, c)$$

Greedy chỉ nhìn transition hiện tại.

Beam search

- Giữ nhiều configuration tốt nhất.
- Danh sách này gọi là **agenda**.
- Số lượng giữ lại gọi là **beam width**.

$$\text{ConfigScore}(c_i) =$$

$$\text{ConfigScore}(c_{i-1}) + \text{Score}(t_i, c_{i-1})$$

Beam search tính điểm theo cả lịch sử transition.

Thuật toán beam search

function DEPENDENCYBEAMPARSE(*words*, *width*) **returns** dependency tree

state $\leftarrow$ {[root], [*words*], [], 0.0} ;initial configuration

agenda $\leftarrow$ (*state*) ;initial agenda

while *agenda* **contains** non-final states

newagenda $\leftarrow$ {}

for each *state* $\in$ *agenda* **do**

for all {*t* | *t* $\in$ VALIDOPERATORS(*state*)} **do**

child $\leftarrow$ APPLY(*t*, *state*)

newagenda $\leftarrow$ ADDBEAM(*child*, *newagenda*, *width*)

agenda $\leftarrow$ *newagenda*

return BESTOF(*agenda*)

function ADDBEAM(*state*, *agenda*, *width*) **returns** updated agenda

if LENGTH(*agenda*) < *width* **then**

agenda $\leftarrow$ INSERT(*state*, *agenda*)

else if SCORE(*state*) > SCORE(WORSTOF(*agenda*))

agenda $\leftarrow$ REMOVE(WORSTOF(*agenda*))

agenda $\leftarrow$ INSERT(*state*, *agenda*)

return *agenda*

Luồng xử lý

- 1 Khởi tạo state đầu tiên.
- 2 Đưa state vào agenda.
- 3 Với mỗi state, thử các transition hợp lệ.
- 4 Tạo child state mới.
- 5 Chỉ giữ các state tốt nhất theo beam width.
- 6 Trả về state tốt nhất cuối cùng.

Figure 19.10 Beam search applied to transition-based dependency parsing.

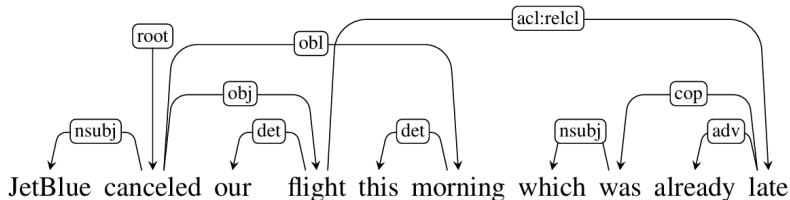
Hình 19.10: Beam search cho transition-based parsing

Table of Contents

- 1 19.2.2 Feature-Based Classifier
- 2 19.2.3 Neural Classifier
- 3 19.2.4 Advanced Methods in Transition-Based Parsing
- 4 **19.3 Graph-Based Dependency Parsing**
 - Parsing via finding the maximum spanning tree
 - A feature-based algorithm for assigning scores
 - Neural Algorithm for Assigning Scores
- 5 19.4 Evaluation
- 6 19.5 Summary

Graph-Based Dependency Parsing

- Độ chính xác thường cao hơn transition-based parsers
- Xử lý tốt quan hệ phụ thuộc khoảng cách xa
- Dựa trên **global scoring** thay vì quyết định cục bộ
- Có thể tạo **non-projective trees**
- Quan trọng với nhiều ngôn ngữ trên thế giới



Ý tưởng chính

Tìm cây phụ thuộc tốt nhất trong không gian tất cả các cây khả thi:

$$\hat{T}(S) = \arg \max_{t \in G_S} \text{Score}(t, S)$$

- S : câu đầu vào
- t : cây ứng viên
- G_S : tập các cây khả thi
- $\text{Score}(t, S)$: điểm số của cây

Graph-Based Dependency Parsing

Giả định tính điểm

Điểm của cây bằng tổng điểm các cạnh:

$$Score(t, S) = \sum_{e \in t} Score(e)$$

Ý nghĩa:

- Đánh giá từng dependency edge
- Cộng tổng điểm toàn bộ cây
- Chọn cây có điểm cao nhất

Graph-Based Dependency Parsing

Hai bài toán chính

- 1 Gán score cho từng edge
- 2 Tìm cây parse tối ưu

Các hướng giải quyết

- Feature-based algorithms
- Neural scoring models

Xây dựng graph G

- $G = (V, E)$
- V : tập từ + ROOT
- E : tất cả cạnh có hướng giữa các cặp node
- ROOT $\rightarrow$ mọi từ trong câu

Parsing via finding the maximum spanning tree

Xây dựng graph G

- $G = (V, E)$
- V : tập từ + ROOT
- E : tất cả cạnh có hướng giữa các cặp node
- ROOT $\rightarrow$ mọi từ trong câu

Ví dụ

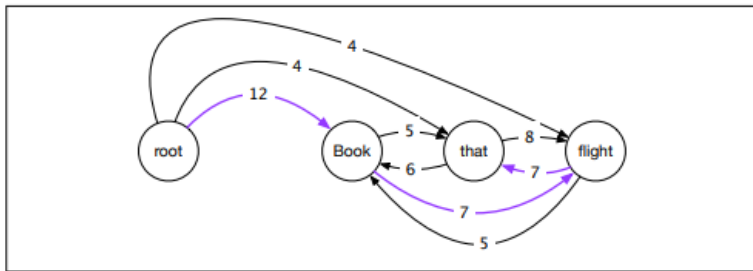


Figure 19.11 Initial rooted, directed graph for *Book that flight*.

Parsing via finding the maximum spanning tree

Trực giác 1

- Mỗi node có đúng 1 cạnh đi vào

Trực giác 1

- Mỗi node có đúng 1 cạnh đi vào
- Riêng node root không có cạnh đi vào
- Thành phần liên thông cũng có 1 cạnh vào

Parsing via finding the maximum spanning tree

Trực giác 1

- Mỗi node có đúng 1 cạnh đi vào
- Riêng node root không có cạnh đi vào
- Thành phần liên thông cũng có 1 cạnh vào

Trực giác 2

- Quan trọng là **relative score**, không phải absolute score

Parsing via finding the maximum spanning tree

Trực giác 1

- Mỗi node có đúng 1 cạnh đi vào
- Riêng node root không có cạnh đi vào
- Thành phần liên thông cũng có 1 cạnh vào

Trực giác 2

- Quan trọng là **relative score**, không phải absolute score
- Trừ cùng 1 hằng số không làm đổi MST
- Vì mọi cây đều bị trừ như nhau

Parsing via finding the maximum spanning tree

Greedy idea

- Với mỗi node: chọn cạnh vào có score cao nhất
- Nếu tạo thành cây → done
- Đây là lời giải tối ưu trong trường hợp không có chu trình

Parsing via finding the maximum spanning tree

Greedy idea

- Với mỗi node: chọn cạnh vào có score cao nhất
- Nếu tạo thành cây → done
- Đây là lời giải tối ưu trong trường hợp không có chu trình

Vấn đề của greedy

Parsing via finding the maximum spanning tree

Greedy idea

- Với mỗi node: chọn cạnh vào có score cao nhất
- Nếu tạo thành cây → done
- Đây là lời giải tối ưu trong trường hợp không có chu trình

Vấn đề của greedy

- Có thể sinh ra chu trình (cycle)
- Khi đó không còn là cây khung hợp lệ
- Cần thuật toán MST chuẩn để xử lý

Parsing via finding the maximum spanning tree

Greedy idea

- Với mỗi node: chọn cạnh vào có score cao nhất
- Nếu tạo thành cây → done
- Đây là lời giải tối ưu trong trường hợp không có chu trình

Vấn đề của greedy

- Có thể sinh ra chu trình (cycle)
- Khi đó không còn là cây khung hợp lệ
- Cần thuật toán MST chuẩn để xử lý

Giải pháp: Thuật toán Chu-Liu Edmonds

Chu-Liu-Edmonds (CLE Algorithm)

function MAXSPANNINGTREE($G=(V,E)$, $root$, $score$) **returns** spanning tree

```

 $F \leftarrow []$ 
 $T' \leftarrow []$ 
 $score' \leftarrow []$ 
for each  $v \in V$  do
     $bestInEdge \leftarrow \operatorname{argmax}_{e=(u,v) \in E} score[e]$ 
     $F \leftarrow F \cup bestInEdge$ 
    for each  $e=(u,v) \in E$  do
         $score'[e] \leftarrow score[e] - score[bestInEdge]$ 

if  $T=(V,F)$  is a spanning tree then return it
else
     $C \leftarrow$  a cycle in  $F$ 
     $G' \leftarrow \operatorname{CONTRACT}(G, C)$ 
     $T' \leftarrow \operatorname{MAXSPANNINGTREE}(G', root, score')$ 
     $T \leftarrow \operatorname{EXPAND}(T', C)$ 
return  $T$ 
    
```

function CONTRACT(G, C) **returns** contracted graph

function EXPAND(T, C) **returns** expanded graph

Figure 19.12 The Chu-Liu Edmonds algorithm for finding a maximum spanning tree in a weighted directed graph.

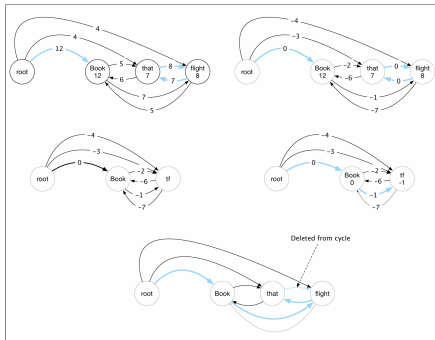


Figure 19.13 Chu-Liu-Edmonds graph-based example for Book that flight

Chu-Liu-Edmonds (CLE Algorithm)

function MAXSPANNINGTREE($G=(V,E)$, $root$, $score$) **returns** spanning tree

```

 $F \leftarrow []$ 
 $T' \leftarrow []$ 
 $score' \leftarrow []$ 
for each  $v \in V$  do
   $bestInEdge \leftarrow \operatorname{argmax}_{e=(u,v) \in E} score[e]$ 
   $F \leftarrow F \cup bestInEdge$ 
  for each  $e=(u,v) \in E$  do
     $score'[e] \leftarrow score[e] - score[bestInEdge]$ 

if  $T=(V,F)$  is a spanning tree then return it
else
   $C \leftarrow$  a cycle in  $F$ 
   $G' \leftarrow \text{CONTRACT}(G, C)$ 
   $T' \leftarrow \text{MAXSPANNINGTREE}(G', root, score')$ 
   $T \leftarrow \text{EXPAND}(T', C)$ 
return  $T$ 

```

function CONTRACT(G, C) **returns** contracted graph

function EXPAND(T, C) **returns** expanded graph

Figure 19.12 The Chu-Liu Edmonds algorithm for finding a maximum spanning tree in a weighted directed graph.

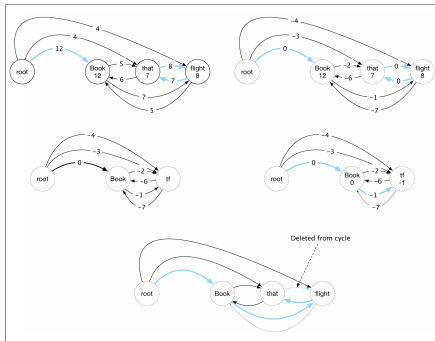


Figure 19.13 Chu-Liu-Edmonds graph-based example for *Book that flight*

- Độ phức tạp tổng quát: $O(mn)$
- Độ phức tạp đối với đồ thị kết nối đầy đủ: $O(n^3)$

Gabow và cộng sự đã đề xuất phiên bản tối ưu hơn với độ phức tạp $O(m + n \log(n))$

A feature-based algorithm for assigning scores

- Score của cây = tổng score các cạnh

$$score(S, T) = \sum_{e \in T} score(S, e)$$

- Score mỗi cạnh = tổ hợp tuyến tính các feature

$$score(S, e) = \sum_{i=1}^N w_i f_i(S, e) = w \cdot f$$

- f_i : đặc trưng của cạnh
- w_i : trọng số cần học
- Cần: chọn feature + huấn luyện weights

A feature-based algorithm for assigning scores

Feature examples

- Wordform, lemma, POS của head & dependent
- Context: từ trước / sau / giữa
- Word embeddings
- Dependency relation
- Direction (left / right)
- Distance giữa head và dependent

A feature-based algorithm for assigning scores

- Mục tiêu: học trọng số w cho feature
- Tối ưu: $\text{score}(\text{cây đúng}) > \text{score}(\text{cây sai})$
- Không phải classification hay action prediction

Perceptron-based learning

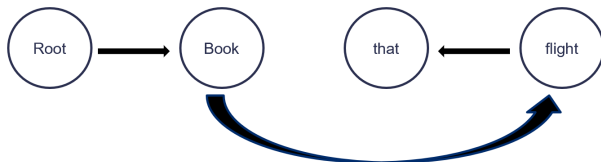
- Parse một câu bằng một tập weights được khởi tạo ngẫu nhiên
- So sánh với gold tree
- Nếu sai $\rightarrow$ cập nhật weights

Update rule

- Giảm weight feature của parse sai
- Cập nhật theo **learning rate**
- Lặp đến khi hội tụ

A feature-based algorithm for assigning scores

Gold tree:



Predicted tree:



Cách cập nhật trọng số

Câu ví dụ: “Book that flight”. Gold tree: root -> book -> flight -> that
Dự đoán: root -> book -> that -> flight

<u>Tên features</u>	Predicted tree	Gold tree	<u>Cập nhật trọng số</u>
(root, Book)	<u>Có</u>	<u>Có</u>	
(Book, flight)	<u>Không</u>	<u>Có</u>	
(flight, that)	<u>Không</u>	<u>Có</u>	
(Book, that)	<u>Có</u>	<u>Không</u>	

Cách cập nhật trọng số

Câu ví dụ: “Book that flight”. Gold tree: root -> book -> flight -> that

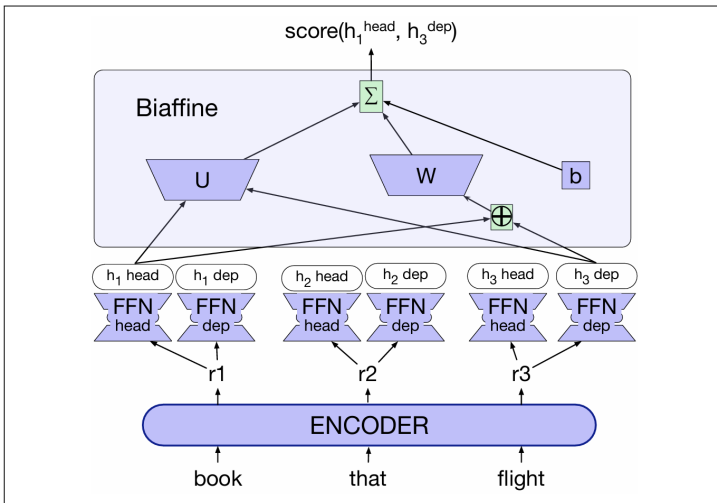
Dự đoán: root -> book -> that -> flight

<u>Tên features</u>	Predicted tree	Gold tree	<u>Cập nhật trọng số</u>
(root, Book)	<u>Có</u>	<u>Có</u>	w_t
(Book, flight)	<u>Không</u>	<u>Có</u>	$w_t + \alpha$
(flight, that)	<u>Không</u>	<u>Có</u>	$w_t + \alpha$
(Book, that)	<u>Có</u>	<u>Không</u>	$w_t - \alpha$

Ý tưởng

- Graph-based multilingual parsers hiện đại dùng **neural networks**
- Thay vì feature thủ công cho từng edge (w_i, w_j)
- Câu được đưa qua **encoder** để tạo biểu diễn cho từng từ
- Kết hợp embedding của (w_i, w_j) để tính score cho cạnh $i \rightarrow j$
- Score được dự đoán trực tiếp bởi neural network

Neural Algorithm for Assigning Score



Neural Algorithm for Assigning Score

- Input câu: $X = x_1, \dots, x_n$
- Encoder $\rightarrow$ contextual embedding: $R = r_1, \dots, r_n$

Head / Dep representation

$$h_i^{head} = FFN_{head}(r_i)$$

$$h_i^{dep} = FFN_{dep}(r_i)$$

Neural Algorithm for Assigning Score

- Input câu: $X = x_1, \dots, x_n$
- Encoder $\rightarrow$ contextual embedding: $R = r_1, \dots, r_n$

Head / Dep representation

$$h_i^{head} = FFN_{head}(r_i)$$

$$h_i^{dep} = FFN_{dep}(r_i)$$

Edge scoring

Neural Algorithm for Assigning Score

- Input câu: $X = x_1, \dots, x_n$
- Encoder $\rightarrow$ contextual embedding: $R = r_1, \dots, r_n$

Head / Dep representation

$$h_i^{head} = FFN_{head}(r_i)$$

$$h_i^{dep} = FFN_{dep}(r_i)$$

Edge scoring

$$Score(i \rightarrow j) = Biaff(h_i^{head}, h_j^{dep})$$

$$Biaff(x, y) = x^T U y + W(x \oplus y) + b$$

U, W, b là tham số học được; biaffine cho phép học multiplicative interactions giữa vectors.

Neural Algorithm for Assigning Score

- Ta tính score cho mỗi cặp $(i \rightarrow j)$:

$$Score(i \rightarrow j)$$

- Chuẩn hóa bằng softmax:

$$p(i \rightarrow j) = \text{softmax}([Score(k \rightarrow j) ; \forall k \neq j, 1 \leq k \leq n])$$

- Xác suất này được đưa vào thuật toán MST để tìm cây phụ thuộc tốt nhất.
- Bộ phân loại $p(i \rightarrow j)$ được huấn luyện bằng cách tối ưu hàm mất mát **cross-entropy loss**.

Neural Algorithm for Assigning Score

'Dozat & Manning (2017) Dependency Parsing' (Thuật toán Phân tích Cú pháp Dozat & Manning (2017))

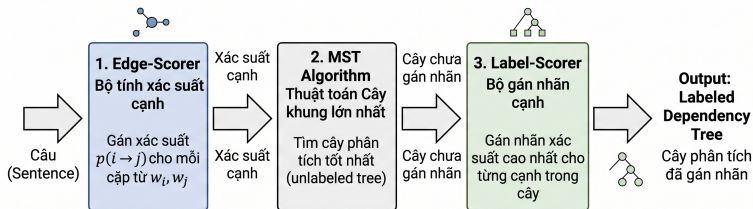
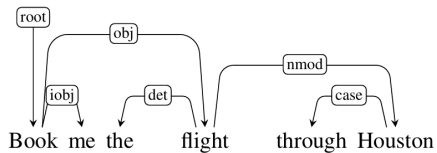


Table of Contents

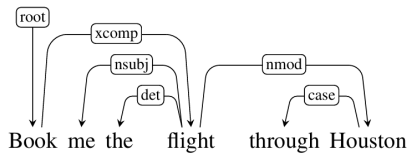
- 1 19.2.2 Feature-Based Classifier
- 2 19.2.3 Neural Classifier
- 3 19.2.4 Advanced Methods in Transition-Based Parsing
- 4 19.3 Graph-Based Dependency Parsing
 - Parsing via finding the maximum spanning tree
 - A feature-based algorithm for assigning scores
 - Neural Algorithm for Assigning Scores
- 5 19.4 Evaluation
- 6 19.5 Summary

- **Test set evaluation:** giống như phrase-structure parsing
- **Exact Match (EM):**
 - số câu parse đúng hoàn toàn
 - quá khắt khe, không đủ chi tiết để đánh giá cải tiến
- **Labeled / Unlabeled Attachment**
 - **LAS:** đúng head + đúng dependency label
 - **UAS:** đúng head, bỏ qua label
 - Accuracy = % token gán đúng head (và label nếu LAS)
- **Label Accuracy Score (LS):**
 - % token có label đúng, bỏ qua head

Evaluation

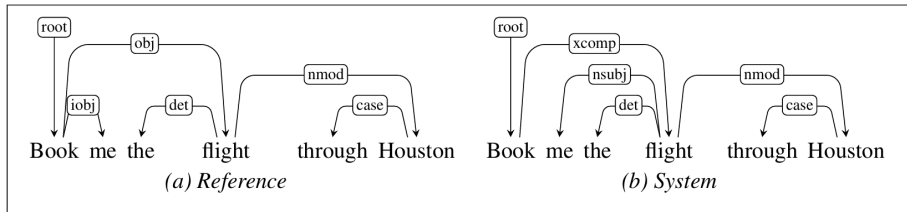


(a) Reference



(b) System

Evaluation



- $LAS = \frac{4}{6}$
- $UAS = \frac{5}{6}$

- Ngoài attachment score, đánh giá theo từng quan hệ phụ thuộc (vd: **NSUBJ**)
- Trên development corpus
- **Precision:**
 - % quan hệ NSUBJ dự đoán là đúng
- **Recall:**
 - % quan hệ NSUBJ thật được phát hiện
- **Confusion matrix:**
 - theo dõi mức độ nhầm lẫn giữa các dependency label

Table of Contents

- 1 19.2.2 Feature-Based Classifier
- 2 19.2.3 Neural Classifier
- 3 19.2.4 Advanced Methods in Transition-Based Parsing
- 4 19.3 Graph-Based Dependency Parsing
 - Parsing via finding the maximum spanning tree
 - A feature-based algorithm for assigning scores
 - Neural Algorithm for Assigning Scores
- 5 19.4 Evaluation
- 6 19.5 Summary

Summary

- Cấu trúc câu = tập quan hệ nhị phân giữa các từ; không mã hóa constituency
- Quan hệ dependency thể hiện head-dependent giữa các từ
- Hữu ích cho IE, semantic parsing, question answering
- Transition-based parsing: greedy, dựa trên stack
- Graph-based parsing: dùng Maximum Spanning Tree (MST)
- Cả hai hướng đều dùng supervised learning
- Treebank là dữ liệu huấn luyện (manual hoặc chuyển đổi từ phrase-structure)
- Đánh giá bằng LAS/UAS trên dev set và test set riêng biệt

CẢM ƠN CÁC BẠN!

Nhóm 3

QUIZ TIME!

Mời các bạn tham gia quizz để ôn lại kiến thức.